

PROGRAMMING

Hadia Naseer

April 2024

1 CHAPTER 5

1.1 FACTORIAL FUNCTION

```
#include <stdio.h>

// Function to calculate factorial using recursion
int factorial(int n) {
    if (n == 0) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

int main() {
    int num;

    printf("Enter a non-negative integer: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        printf("Factorial of %d is %d\n", num, factorial(num));
    }

    return 0;
}
```

EXPLANATION

the factorial function computes the factorial of a non-negative integer n recursively. The base case is when n is 0, in which case the factorial is 1. For any other value of n , the function recursively calls itself with $n-1$ and multiplies the

result by n . In the main function, the user inputs a non-negative integer. The factorial function is called with the input number as an argument to compute its factorial. Finally, the program prints the factorial of the input number. Now, let's perform a dry run of the program with an example:

Suppose the user inputs $\text{num} = 5$:

The program starts execution. It prompts the user to input a non-negative integer, and the user enters 5. In the main function, it calls the factorial function with $\text{num} = 5$. Inside the factorial function: Since n is not 0, it recursively calls itself with $n-1$, i.e., $\text{factorial}(4)$. Inside $\text{factorial}(4)$: It recursively calls itself with $n-1$, i.e., $\text{factorial}(3)$. Inside $\text{factorial}(3)$: It recursively calls itself with $n-1$, i.e., $\text{factorial}(2)$. Inside $\text{factorial}(2)$: It recursively calls itself with $n-1$, i.e., $\text{factorial}(1)$. Inside $\text{factorial}(1)$: It recursively calls itself with $n-1$, i.e., $\text{factorial}(0)$. Since n is 0, it returns 1. $\text{factorial}(1)$ returns 1. $\text{factorial}(2)$ returns $2 * 1 = 2$. $\text{factorial}(3)$ returns $3 * 2 = 6$. $\text{factorial}(4)$ returns $4 * 6 = 24$. Finally, the factorial function in the main function returns $5 * 24 = 120$. The program prints "Factorial of 5 is 120".

1.2 FIBBONACI FUNCTION

```
#include <stdio.h>

// Function to calculate Fibonacci series using recursion
int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n, i;

    printf("Enter the number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci Series: ");
    for (i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }

    return 0;
}
```

EXPLANATION In this program, we have a function `fibonacci` which computes the n th Fibonacci number recursively. The base cases for the Fibonacci

series are $\text{fib}(0) = 0$ and $\text{fib}(1) = 1$. For any other value of n , the function recursively calls itself with $n-1$ and $n-2$, and returns the sum of the results. In the main function, we input the number of terms in the Fibonacci series from the user. We then print the Fibonacci series using a loop, calling the fibonacci function for each term and printing the result. Now, let's perform a dry run of the program with an example:

Suppose the user inputs $n = 6$:

The program starts execution. It prompts the user to input the number of terms, and the user enters 6. In the main function, it enters a loop from $i = 0$ to $i = n-1 = 5$. For each value of i in the loop, it calls the fibonacci function with the value of i and prints the result. For $i = 0$, it calls `fibonacci(0)`, which returns 0. For $i = 1$, it calls `fibonacci(1)`, which returns 1. For $i = 2$, it calls `fibonacci(2)`. This recursively calls `fibonacci(1)` and `fibonacci(0)`. `fibonacci(1)` returns 1 and `fibonacci(0)` returns 0. So, `fibonacci(2)` returns $1 + 0 = 1$. Similarly, for $i = 3$, it calls `fibonacci(3)`, which recursively calls `fibonacci(2)` and `fibonacci(1)`. `fibonacci(2)` returns 1 and `fibonacci(1)` returns 1. So, `fibonacci(3)` returns $1 + 1 = 2$. This process continues until $i = 5$, where it calls `fibonacci(5)`. This recursively calls `fibonacci(4)` and `fibonacci(3)`. `fibonacci(4)` returns 3 and `fibonacci(3)` returns 2. So, `fibonacci(5)` returns $3 + 2 = 5$. Finally, it prints the Fibonacci series: 0 1 1 2 3 5.

1.3 SUM OF TWO INTEGERS

```
#include <stdio.h>

int main() {
    int num1, num2, sum;

    printf("Enter first integer: ");
    scanf("%d", &num1);

    printf("Enter second integer: ");
    scanf("%d", &num2);

    sum = num1 + num2;

    printf("Sum of %d and %d is %d\n", num1, num2, sum);

    return 0;
}
```

EXPLANATION The sum function computes the sum of two integers recursively. The base case checks if one of the integers is zero, in which case it returns the other integer. For any other case, the function recursively calls itself with one of the integers reduced by 1, and adds 1 to the result. In the main function,

the user inputs two integers. The sum function is called with the input integers as arguments to compute their sum. Finally, the program prints the sum of the input integers. Now, let's perform a dry run of the program with an example:

Suppose the user inputs $\text{num1} = 3$ and $\text{num2} = 4$:

The program starts execution. It prompts the user to input two integers, and the user enters 3 and 4. In the main function, it calls the sum function with $\text{num1} = 3$ and $\text{num2} = 4$. Inside the sum function: Since both integers are not zero, it recursively calls itself with $\text{num1} = 2$ and $\text{num2} = 4$. Inside the next recursive call: It recursively calls itself with $\text{num1} = 1$ and $\text{num2} = 4$. Inside the next recursive call: It recursively calls itself with $\text{num1} = 0$ and $\text{num2} = 4$. Since num1 is zero, it returns num2 , which is 4. The next level returns $\text{num2} + 1$, which is $4 + 1 = 5$. The top level returns $\text{num2} + 1$, which is $5 + 1 = 6$. Finally, the sum function in the main function returns 6. The program prints "Sum of 3 and 4 is 6".

1.4 GREATEST COMMON DIVISOR

```
#include <stdio.h>

// Function to compute the greatest common divisor (GCD) of two integers recursively
int gcd(int a, int b) {
    // Base case: if one of the numbers is zero, return the other number
    if (b == 0) {
        return a;
    } else {
        // Recursive case: call gcd with b and the remainder of a divided by b
        return gcd(b, a % b);
    }
}

int main() {
    int num1, num2;

    // Input the two integers
    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);

    // Compute the GCD using the function
    int result = gcd(num1, num2);

    // Display the result
    printf("Greatest Common Divisor of %d and %d is %d\n", num1, num2, result);

    return 0;
}
```

EXPLANATION The gcd function computes the greatest common divisor (GCD) of two integers recursively using the Euclidean algorithm. The base case checks if one of the numbers is zero. If so, it returns the other number, as the GCD of any number and zero is the number itself. For any other case, the function recursively calls itself with the second number and the remainder of the first number divided by the second number. In the main function, the user inputs two integers. The gcd function is called with the input integers as arguments to compute their GCD. Finally, the program prints the GCD of the input integers. Now, let's perform a dry run of the program with an example:

Suppose the user inputs num1 = 24 and num2 = 18:

The program starts execution. It prompts the user to input two integers, and the user enters 24 and 18. In the main function, it calls the gcd function with num1 = 24 and num2 = 18. Inside the gcd function: Since b is not zero, it recursively calls itself with b = 18 and the remainder of 24 / 18, which is 6. Inside the next recursive call: Since b is not zero, it recursively calls itself with b = 6 and the remainder of 18 / 6, which is 0. Since b is now zero, it returns a, which is 6. The top level returns 6. Finally, the gcd function in the main function returns 6. The program prints "Greatest Common Divisor of 24 and 18 is 6".

1.5 MULTIPLY TWO INTEGERS

```
#include <stdio.h>

// Function to calculate multiplication using recursion
int multiply(int a, int b) {
    if (b == 0) {
        return 0;
    } else {
        return a + multiply(a, b - 1);
    }
}

int main() {
    int num1, num2;

    printf("Enter first integer: ");
    scanf("%d", &num1);

    printf("Enter second integer: ");
    scanf("%d", &num2);

    printf("Product of %d and %d is %d\n", num1, num2, multiply(num1, num2));

    return 0;
}
```

EXPLANATION Initialization:

We start by declaring variables num1, num2, and result. We also declare a function product that takes two integer parameters a and b and returns their product. Input:

The program prompts the user to enter two numbers. Let's assume the user inputs 3 and 5. Function Call:

The product function is called with the arguments 3 and 5. Inside the function, a is assigned the value 3 and b is assigned the value 5. The function computes the product of a and b (i.e., $3 * 5$) and returns the result, which is 15. Result Assignment:

The returned value from the product function (i.e., 15) is stored in the variable result. Output:

The program prints the result using printf. It prints "Product of 3 and 5 is 15" because num1 is 3, num2 is 5, and the result is 15. Termination:

The program terminates after executing all statements in the main function.

1.6 TOWERS OF HANOI

```
#include <stdio.h>

// Function to move disks from source to destination
void towersOfHanoi(int n, char source, char destination, char auxiliary) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", source, destination);
        return;
    }
    towersOfHanoi(n - 1, source, auxiliary, destination);
    printf("Move disk %d from %c to %c\n", n, source, destination);
    towersOfHanoi(n - 1, auxiliary, destination, source);
}

int main() {
    int numDisks;

    printf("Enter the number of disks: ");
    scanf("%d", &numDisks);

    towersOfHanoi(numDisks, 'A', 'C', 'B');

    return 0;
}
```

EXPLANATION The towerOfHanoi function solves the Tower of Hanoi problem recursively. In the base case, if there is only one disk to move ($n == 1$),

it directly moves the disk from the source rod to the destination rod. In the recursive case, it first moves $n-1$ disks from the source rod to the auxiliary rod, using the destination rod as auxiliary. Then, it moves the n th disk from the source rod to the destination rod. Finally, it moves $n-1$ disks from the auxiliary rod to the destination rod, using the source rod as auxiliary. In the main function, it defines the number of disks n and the names of the rods (source, auxiliary, and destination). It then calls the `towerOfHanoi` function with these parameters. Now, let's perform a dry run of the program with $n = 3$:

The program starts execution. It calls the `towerOfHanoi` function with $n = 3$, source rod A, auxiliary rod B, and destination rod C. Inside the `towerOfHanoi` function: It recursively calls itself to move 2 disks from rod A to rod B, using rod C as auxiliary. It prints the move of the 3rd disk from rod A to rod C. It recursively calls itself to move 2 disks from rod B to rod C, using rod A as auxiliary. The program prints the solution step by step.

1.7 RAISING AN INTEGER TO AN INTEGER POWER

```
#include <stdio.h>

// Function to compute the power of a number recursively
int power(int base, int exponent) {
    // Base case: if exponent is 0, return 1
    if (exponent == 0) {
        return 1;
    } else {
        // Recursive case: multiply base by power(base, exponent - 1)
        return base * power(base, exponent - 1);
    }
}

int main() {
    int base, exponent;

    // Input the base and exponent
    printf("Enter the base: ");
    scanf("%d", &base);
    printf("Enter the exponent: ");
    scanf("%d", &exponent);

    // Compute the power using the function
    int result = power(base, exponent);

    // Display the result
    printf("%d raised to the power of %d is %d\n", base, exponent, result);

    return 0;
}
```

```
}
```

EXPLANATION The power function computes the power of a number recursively. The base case checks if the exponent is 0. If so, it returns 1, as any number raised to the power of 0 is 1. For any other case, the function recursively calls itself with the base multiplied by the result of `power(base, exponent - 1)`. In the main function, the user inputs the base and exponent. The power function is called with the input base and exponent as arguments to compute the power. Finally, the program prints the result. Now, let's perform a dry run of the program with an example:

Suppose the user inputs `base = 2` and `exponent = 3`:

The program starts execution. It prompts the user to input the base and exponent, and the user enters 2 and 3. In the main function, it calls the power function with `base = 2` and `exponent = 3`. Inside the power function: Since exponent is not 0, it recursively calls itself with `base = 2` and `exponent - 1 = 2`. Inside the next recursive call: Since exponent is not 0, it recursively calls itself with `base = 2` and `exponent - 1 = 1`. Inside the next recursive call: Since exponent is not 0, it recursively calls itself with `base = 2` and `exponent - 1 = 0`. Since exponent is now 0, it returns 1. The previous level returns `base * 1`, which is $2 * 1 = 2$. The top level returns `base * 2`, which is $2 * 2 = 4$. Finally, the power function in the main function returns 4. The program prints "2 raised to the power of 3 is 4".

1.8 RECURSIVE MAIN

```
#include <stdio.h>

// Recursive function to calculate factorial
int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    } else {
        return n * factorial(n - 1);
    }
}

int main() {
    int num;

    printf("Enter a non-negative integer: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
```



```

        printf("Factorial of %d is %d\n", num, factorial(num));
    }

    return 0;
}

```

EXPLANATION We define a function recursiveMain that simulates recursive behavior. This function takes an integer count as a parameter. In the base case, when count becomes 0, we print "End of recursion" and return. In the recursive case, we print the current value of count, and then recursively call recursiveMain with count - 1. In the main function, we start the recursion by calling recursiveMain with an initial value of 5. Now, let's perform a dry run of the program:

The program starts execution. In the main function, it calls recursiveMain(5). Inside recursiveMain(5), count is not zero, so it prints "Count: 5" and calls recursiveMain(4). Inside recursiveMain(4), count is not zero, so it prints "Count: 4" and calls recursiveMain(3). This process continues until recursiveMain(0), where count becomes 0. At this point, it prints "End of recursion" and returns. The recursion unwinds, and each function call returns to its caller. Finally, the main function returns, and the program terminates. This is how we can achieve a form of recursion within the main function in C by calling another function recursively.

1.9 PRINTING KEYBOARD INPUTS IN REVERSE

```

#include <stdio.h>

// Function to print characters in reverse recursively
void printReverse() {
    char ch = getchar(); // Read a character from the input buffer

    // Base case: if newline character is encountered, return
    if (ch == '\n') {
        return;
    }

    // Recursive case: recursively call printReverse
    printReverse();

    // Print the character after all recursive calls return
    putchar(ch);
}

int main() {
    printf("Enter characters (press Enter when done):\n");
}

```

```

        // Call printReverse to print characters in reverse
        printReverse();

    return 0;
}

```

EXPLANATION We define a function `printReverse` that prints characters in reverse recursively. Inside the function, we read a character from the input buffer using `getchar`. In the base case, if the character is a newline (`"`), we return, as we've reached the end of input. In the recursive case, we recursively call `printReverse`. After all recursive calls return, we print the character using `putchar`. In the main function, we prompt the user to enter characters and then call `printReverse`. Now, let's perform a dry run of the program:

Suppose the user enters "hello" followed by Enter:

The program prompts the user to enter characters. In the main function, it calls `printReverse`. Inside `printReverse`: It reads the character 'h' from the input buffer. It recursively calls `printReverse`. Inside the recursive call, it reads the character 'e' from the input buffer. It recursively calls `printReverse`. Inside the recursive call, it reads the character 'l' from the input buffer. It recursively calls `printReverse`. Inside the recursive call, it reads the character 'l' from the input buffer. It recursively calls `printReverse`. Inside the recursive call, it reads the character 'o' from the input buffer. It recursively calls `printReverse`. Inside the recursive call, it reads the newline character `"` from the input buffer. As the base case is met, the function returns without printing anything. After returning from the recursive call, it prints the character 'o'. After returning from the recursive call, it prints the character 'l'. After returning from the recursive call, it prints the character 'l'. After returning from the recursive call, it prints the character 'e'. After returning from the recursive call, it prints the character 'h'. Finally, the main function returns, and the program terminates.

1.10 VISUALIZING RECURSION

```

#include <stdio.h>

// Recursive function to count down from n to 1
void countDown(int n) {
    // Base case: if n is 0, return
    if (n == 0) {
        printf("Base case reached: n = 0\n");
        return;
    }

    // Recursive case: print n and recursively call countDown with n-1
}

```

```

        printf("n = %d\n", n);
        countDown(n - 1);
    }

    int main() {
        printf("Visualizing recursion:\n");
        countDown(3); // Start with n = 3
        return 0;
    }

```

EXPLANATION The `countDown` function is a recursive function that counts down from a given number `n` to 1. In the base case, if `n` is 0, it prints a message indicating that the base case is reached and returns. In the recursive case, it prints the current value of `n` and then recursively calls `countDown` with `n-1`. In the main function, it calls `countDown` with an initial value of 3. Now, let's perform a dry run of the program:

The program starts execution. In the main function, it calls `countDown(3)`. Inside the `countDown` function: It prints `n = 3` and recursively calls `countDown(2)`. Inside the next recursive call: It prints `n = 2` and recursively calls `countDown(1)`. Inside the next recursive call: It prints `n = 1` and recursively calls `countDown(0)`. Inside the next recursive call: Since `n = 0`, it prints the base case message and returns. Back in the previous call with `n = 1`, it has completed the recursive call, so it returns. Back in the previous call with `n = 2`, it has completed the recursive call, so it returns. Back in the initial call with `n = 3`, it has completed the recursive call, so it returns. The program terminates. During the execution, you can visualize the call stack forming and unwinding as each function call is made and returned, respectively. This illustrates how recursion works by breaking down a problem into smaller subproblems and solving each subproblem using the same recursive approach.

2 CHAPTER 6

2.1 SUM OF ELEMENTS OF AN ARRAY

```

#include <stdio.h>

// Function to compute the sum of elements of an array recursively
int arraySum(int arr[], int size) {
    // Base case: if size is 0, return 0
    if (size == 0) {
        return 0;
    } else {
        // Recursive case: sum the last element with the sum of elements before it
        return arr[size - 1] + arraySum(arr, size - 1);
    }
}

```

```

}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    // Compute the sum of elements using the function
    int result = arraySum(arr, size);

    // Display the result
    printf("Sum of elements of the array is %d\n", result);

    return 0;
}

```

EXPLANATION The arraySum function computes the sum of elements of an array recursively. The base case checks if the size of the array is 0. If so, it returns 0. For any other case, the function recursively calls itself with the last element removed from the array, and adds the last element to the result. In the main function, we define an array arr and its size. We call the arraySum function with the array and its size as arguments to compute the sum of elements. Finally, the program prints the result. Now, let's perform a dry run of the program:

Suppose the array arr contains 1, 2, 3, 4, 5:

The program starts execution. In the main function, it calls the arraySum function with the array arr and size 5. Inside the arraySum function: Since size is not 0, it recursively calls itself with size - 1 = 4. Inside the next recursive call: It recursively calls itself with size - 1 = 3. This process continues until size becomes 0. At this point, the base case is met, so it returns 0. After returning from the recursive calls, it adds the last element (5) to the sum of the previous elements (1 + 2 + 3 + 4). Finally, the arraySum function in the main function returns 15. The program prints "Sum of elements of the array is 15".

2.2 PRINT AN ARRAY

```

#include <stdio.h>

// Function to print elements of an array recursively
void printArray(int arr[], int size) {
    // Base case: if size is 0, return
    if (size == 0) {
        return;
    } else {
        // Recursive case: print the first element and recursively call printArray with size
        printf("%d ", arr[0]);
    }
}

```

```

        printArray(arr + 1, size - 1);
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    // Print the array using the function
    printf("Array: ");
    printArray(arr, size);

    return 0;
}

```

EXPLANATION The printArray function prints elements of an array recursively. The base case checks if the size of the array is 0. If so, it returns. For any other case, the function prints the first element of the array and recursively calls itself with the address of the next element (arr + 1) and size - 1. In the main function, we define an array arr and its size. We call the printArray function with the array and its size as arguments to print the elements of the array. Finally, the program prints the array. Now, let's perform a dry run of the program:

Suppose the array arr contains 1, 2, 3, 4, 5:

The program starts execution. In the main function, it calls the printArray function with the array arr and size 5. Inside the printArray function: Since size is not 0, it prints the first element of the array (1) and recursively calls itself with arr + 1 and size - 1. Inside the next recursive call: It prints the first element of the new array (2) and recursively calls itself with arr + 1 and size - 1. This process continues until size becomes 0. At this point, the base case is met, so it returns. Finally, the program prints "Array: 1 2 3 4 5".

2.3 PRINT AN ARRAY BACKWARD

```

#include <stdio.h>

// Function to print elements of an array backward recursively
void printArrayBackward(int arr[], int size) {
    // Base case: if size is 0, return
    if (size == 0) {
        return;
    } else {
        // Recursive case: recursively call printArrayBackward with size - 1 and print the 1
        printArrayBackward(arr, size - 1);
    }
}

```

```

        printf("%d ", arr[size - 1]);
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    // Print the array backward using the function
    printf("Array backward: ");
    printArrayBackward(arr, size);

    return 0;
}

```

EXPLANATION The `printArrayBackward` function prints elements of an array backward recursively. The base case checks if the size of the array is 0. If so, it returns. For any other case, the function recursively calls itself with `size - 1` to print the elements in reverse order. After the recursive call returns, the function prints the last element of the array. In the main function, we define an array `arr` and its size. We call the `printArrayBackward` function with the array and its size as arguments to print the elements of the array backward. Finally, the program prints the array backward. Now, let's perform a dry run of the program:

Suppose the array `arr` contains 1, 2, 3, 4, 5:

The program starts execution. In the main function, it calls the `printArrayBackward` function with the array `arr` and size 5. Inside the `printArrayBackward` function: Since `size` is not 0, it recursively calls itself with `size - 1` (which is 4). Inside the next recursive call: It recursively calls itself with `size - 1` (which is 3). This process continues until `size` becomes 0. At this point, the base case is met, so it returns without printing anything. After returning from the recursive calls, it prints the last element of the array (5). Finally, the program prints "Array backward: 5 4 3 2 1".

2.4 PRINT A STRING BACKWARD

```

#include <stdio.h>

// Function to print a string backward recursively
void printStringBackward(char *str) {
    // Base case: if the current character is null terminator, return
    if (*str == '\0') {
        return;
    } else {

```

```

        // Recursive case: recursively call printStringBackward with the next character and
        printStringBackward(str + 1);
        printf("%c", *str);
    }
}

int main() {
    char str[] = "Hello, World!";

    // Print the string backward using the function
    printf("String backward: ");
    printStringBackward(str);

    return 0;
}

```

explanation The `printStringBackward` function prints a string backward recursively. The base case checks if the current character is the null terminator (`"`). If so, it returns. For any other case, the function recursively calls itself with the next character (`str + 1`) to print the string in reverse order. After the recursive call returns, the function prints the current character. In the main function, we define a character array `str` containing the string `"Hello, World!"`. We call the `printStringBackward` function with the string as an argument to print the string backward. Finally, the program prints the string backward. Now, let's perform a dry run of the program:

Suppose the string is `"Hello, World!"`:

The program starts execution. In the main function, it calls the `printStringBackward` function with the string `"Hello, World!"`. Inside the `printStringBackward` function: Since the current character is not the null terminator, it recursively calls itself with the next character (`"ello, World!"`) to print the string backward. Inside the next recursive call: It recursively calls itself with the next character (`"llo, World!"`) to print the string backward. This process continues until it reaches the null terminator. At this point, the base case is met, so it returns without printing anything. After returning from the recursive calls, it prints each character backward. Finally, the program prints `"String backward: !dlroW ,olleH"`.

2.5 CHECK WEATHER THE STRING IS PALANDROME

```

#include <stdio.h>
#include <stdbool.h>
#include <string.h>

```

```

// Function to check if a string is a palindrome recursively
bool isPalindrome(char str[], int start, int end) {
    // Base case: if start and end indices cross each other, the string is a palindrome
    if (start >= end) {
        return true;
    }

    // Recursive case: compare characters at start and end indices
    if (str[start] != str[end]) {
        return false;
    }

    // Recursively call isPalindrome with updated indices
    return isPalindrome(str, start + 1, end - 1);
}

int main() {
    char str[100];

    // Input the string
    printf("Enter a string: ");
    scanf("%s", str);

    // Compute the length of the string
    int len = strlen(str);

    // Check if the string is a palindrome using the function
    bool result = isPalindrome(str, 0, len - 1);

    // Display the result
    if (result) {
        printf("%s is a palindrome.\n", str);
    } else {
        printf("%s is not a palindrome.\n", str);
    }

    return 0;
}

```

explanation The `isPalindrome` function checks if a string is a palindrome recursively. The base case checks if the start and end indices cross each other. If so, the string is a palindrome. In the recursive case, the function compares characters at the start and end indices. If they are not equal, the string is not a palindrome. If the characters are equal, the function recursively calls itself with

updated indices to check the remaining part of the string. In the main function, we input a string from the user. We compute the length of the string using the `strlen` function. We call the `isPalindrome` function with the string, start index 0, and end index `len - 1`. Finally, we display whether the string is a palindrome or not based on the result. Now, let's perform a dry run of the program:

Suppose the user inputs the string "racecar":

The program starts execution. It prompts the user to enter a string, and the user enters "racecar". In the main function, it calls the `isPalindrome` function with the string "racecar", start index 0, and end index 6. Inside the `isPalindrome` function: It compares the characters at indices 0 and 6, which are 'r' and 'r', so it proceeds. It recursively calls itself with start index 1 and end index 5. Inside the next recursive call: It compares the characters at indices 1 and 5, which are 'a' and 'a', so it proceeds. It recursively calls itself with start index 2 and end index 4. Inside the next recursive call: It compares the characters at indices 2 and 4, which are 'c' and 'c', so it proceeds.

2.6 MINIMUM VALUE IN AN ARRAY

```
#include <stdio.h>

// Function to find the minimum value in an array recursively
int findMin(int arr[], int size) {
    // Base case: if size is 1, return the only element
    if (size == 1) {
        return arr[0];
    } else {
        // Recursive case: find the minimum of the first element and the minimum of the rest
        int minRest = findMin(arr + 1, size - 1);
        return (arr[0] < minRest) ? arr[0] : minRest;
    }
}

int main() {
    int arr[] = {3, 1, 4, 2, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    // Find the minimum value in the array using the function
    int min = findMin(arr, size);

    // Display the result
    printf("Minimum value in the array: %d\n", min);

    return 0;
}
```

EXPLANATION The findMin function finds the minimum value in an array recursively. The base case checks if the size of the array is 1. If so, it returns the only element in the array. In the recursive case, the function finds the minimum of the first element and the minimum of the rest of the array. We recursively call findMin with the array shifted by one position ($\text{arr} + 1$) and the size decremented by 1. Finally, we return the minimum of the first element and the minimum of the rest of the array. In the main function, we define an array arr and its size. We call the findMin function with the array and its size as arguments to find the minimum value in the array. Finally, we display the minimum value. Now, let's perform a dry run of the program:

Suppose the array arr contains 3, 1, 4, 2, 5:

The program starts execution. In the main function, it calls the findMin function with the array arr and size 5. Inside the findMin function: It recursively calls itself with the array shifted by one position and size decremented by 1. This process continues until the size becomes 1. At this point, the base case is met, and it returns the only element. After returning from the recursive calls, it compares the first element with the minimum of the rest of the array. Finally, the main function prints "Minimum value in the array: 1".

2.7 LINEAR SEARCH

```
#include <stdio.h>

// Function to perform linear search recursively
int linearSearch(int arr[], int size, int key, int index) {
    // Base case: if index exceeds size, key not found
    if (index >= size) {
        return -1;
    }

    // If the key is found at current index, return the index
    if (arr[index] == key) {
        return index;
    }

    // Recursive case: search in the rest of the array
    return linearSearch(arr, size, key, index + 1);
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
    int key = 3;

    // Perform linear search using the function
    int index = linearSearch(arr, size, key, 0);
```

```

    // Display the result
    if (index != -1) {
        printf("Element found at index: %d\n", index);
    } else {
        printf("Element not found in the array.\n");
    }

    return 0;
}

```

EXPLANATION The `linearSearch` function performs linear search recursively. The function takes an array `arr`, its size `size`, the search key, and the current index. The base case checks if the index exceeds size. If so, the key is not found, and -1 is returned. If the key is found at the current index, the index is returned. In the recursive case, the function searches in the rest of the array by recursively calling itself with `index + 1`. In the main function, we define an array `arr`, its size `size`, and the search key. We call the `linearSearch` function with the array, size, key, and initial index 0. Finally, we display whether the element is found and its index if found, or print a message if not found. Now, let's perform a dry run of the program:

Suppose the array `arr` contains 1, 2, 3, 4, 5, and the key to search is 3:

The program starts execution. In the main function, it calls the `linearSearch` function with the array `arr`, size 5, key 3, and index 0. Inside the `linearSearch` function: It checks if the key is found at index 0. Since it's not, it recursively calls itself with index 1. It checks if the key is found at index 1. Since it's not, it recursively calls itself with index 2. It checks if the key is found at index 2. Since it's found, it returns 2. Finally, the main function prints "Element found at index: 2".

2.8 BINARY SEARCH

```

#include <stdio.h>

// Function to perform binary search recursively
int binarySearch(int arr[], int low, int high, int key) {
    // Base case: if low becomes greater than high, key not found
    if (low > high) {
        return -1;
    }

    // Calculate mid index
    int mid = low + (high - low) / 2;

    // If key is present at mid, return mid

```

```

    if (arr[mid] == key) {
        return mid;
    }

    // If key is smaller than mid, search in the left subarray
    if (key < arr[mid]) {
        return binarySearch(arr, low, mid - 1, key);
    }

    // If key is greater than mid, search in the right subarray
    return binarySearch(arr, mid + 1, high, key);
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);
    int key = 3;

    // Perform binary search using the function
    int index = binarySearch(arr, 0, size - 1, key);

    // Display the result
    if (index != -1) {
        printf("Element found at index: %d\n", index);
    } else {
        printf("Element not found in the array.\n");
    }

    return 0;
}

```

EXPLANATION The `binarySearch` function performs binary search recursively. It takes an array `arr`, the lower bound `low`, the upper bound `high`, and the search key. The base case checks if `low` becomes greater than `high`. If so, the key is not found, and `-1` is returned. It calculates the mid index using $(low + high) / 2$. If the key is found at the mid index, it returns the mid index. If the key is smaller than the mid element, it searches in the left subarray by recursively calling itself with `low` and `mid - 1`. If the key is greater than the mid element, it searches in the right subarray by recursively calling itself with `mid + 1` and `high`. In the main function, we define an array `arr`, its size `size`, and the search key. We call the `binarySearch` function with the array, lower bound `0`, upper bound `size - 1`, and the key to search. Finally, we display whether the element is found and its index if found, or print a message if not found. Now, let's perform a dry run of the program:

Suppose the array `arr` contains 1, 2, 3, 4, 5, and the key to search is 3:

The program starts execution. In the main function, it calls the `binarySearch` function with the array `arr`, lower bound 0, upper bound 4, and the key 3. Inside the `binarySearch` function: It calculates the mid index as $(0 + 4) / 2 = 2$, and checks if the key is found at index 2. Since it is found, it returns 2. Finally, the main function prints "Element found at index: 2".

2.9 EIGHT QUEENS

```
#include <stdio.h>
#include <stdbool.h>

#define N 8

// Function to print the board
void printBoard(int board[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%c ", board[i][j] ? 'Q' : '.');
        }
        printf("\n");
    }
    printf("\n");
}

// Function to check if it's safe to place a queen at board[row][col]
bool isSafe(int board[N][N], int row, int col) {
    int i, j;

    // Check left side of this row
    for (i = 0; i < col; i++) {
        if (board[row][i]) {
            return false;
        }
    }

    // Check upper diagonal on left side
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j]) {
            return false;
        }
    }

    // Check lower diagonal on left side
    for (i = row, j = col; j >= 0 && i < N; i++, j--) {
        if (board[i][j]) {
```

```

        return false;
    }
}

return true;
}

// Recursive function to solve the Eight Queens problem
bool solveNQueens(int board[N][N], int col) {
    // Base case: if all queens are placed, return true
    if (col >= N) {
        return true;
    }

    // Consider this column and try placing this queen in all rows
    for (int i = 0; i < N; i++) {
        if (isSafe(board, i, col)) {
            // Place this queen in board[i][col]
            board[i][col] = 1;

            // Recur to place the rest of the queens
            if (solveNQueens(board, col + 1)) {
                return true;
            }

            // If placing queen at board[i][col] doesn't lead to a solution, remove it
            board[i][col] = 0;
        }
    }

    // If the queen cannot be placed in any row in this column, return false
    return false;
}

int main() {
    int board[N][N] = {{0}};

    // Solve the Eight Queens problem
    if (solveNQueens(board, 0)) {
        printf("Solution:\n");
        printBoard(board);
    } else {
        printf("Solution does not exist!\n");
    }

    return 0;
}

```

```
}
```

EXPLANATION The printBoard function prints the chessboard configuration. The isSafe function checks if it's safe to place a queen at a given position on the board. The solveNQueens function is a recursive function to solve the Eight Queens problem. It places queens column by column recursively. In each column, it tries to place a queen in every row and then recursively solves the subproblem. If the subproblem is solvable, it returns true. Otherwise, it backtracks and tries the next row. In the main function, we initialize the board with all zeros and call the solveNQueens function with the initial column 0. If a solution exists, it prints the solution; otherwise, it prints that a solution doesn't exist. Now, let's perform a dry run of the program:

The program starts execution. It calls the solveNQueens function with the initial column 0. Inside the solveNQueens function: It tries to place a queen in each row of the first column. For each position, it checks if it's safe to place the queen. If it's safe, it places the queen and recursively calls itself with the next column. If the subproblem is solvable, it returns true. If not, it backtracks and tries the next row. It continues this process recursively until all queens are placed or it exhausts all possibilities. Finally, if a solution is found, it prints the solution; otherwise, it prints that a solution doesn't exist.

3 CHAPTER 7

3.1 MAZE TRAVERSAL

```
#include <stdio.h>
#include <stdbool.h>

#define N 5 // Size of the maze

// Function to print the maze
void printMaze(int maze[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", maze[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

// Function to check if a cell is valid to move
bool isValidMove(int maze[N][N], int x, int y) {
    return (x >= 0 && x < N && y >= 0 && y < N && maze[x][y] == 1);
}
```

```

}

// Recursive function to solve the maze
bool solveMaze(int maze[N][N], int x, int y, int sol[N][N]) {
    // Base case: if destination is reached
    if (x == N - 1 && y == N - 1) {
        sol[x][y] = 1;
        return true;
    }

    // Check if current cell is valid
    if (isValidMove(maze, x, y)) {
        // Mark the current cell as part of solution path
        sol[x][y] = 1;

        // Move right
        if (solveMaze(maze, x, y + 1, sol)) {
            return true;
        }

        // Move down
        if (solveMaze(maze, x + 1, y, sol)) {
            return true;
        }

        // If none of the above moves work, backtrack
        sol[x][y] = 0;
        return false;
    }

    return false;
}

int main() {
    int maze[N][N] = {
        {1, 0, 0, 0, 0},
        {1, 1, 0, 1, 0},
        {0, 1, 1, 1, 0},
        {0, 0, 0, 1, 0},
        {0, 0, 0, 1, 1}
    };

    // Solution matrix to store the path
    int sol[N][N] = {{0}};

    // Solve the maze

```



```

    if (solveMaze(maze, 0, 0, sol)) {
        printf("Solution found:\n");
        printMaze(sol);
    } else {
        printf("Solution does not exist!\n");
    }

    return 0;
}

```

EXPLANATION The printMaze function prints the maze. The isValidMove function checks if a cell is valid to move. The solveMaze function is a recursive function to solve the maze. It tries moving right and down from the current cell recursively until the destination is reached. In the main function, we define the maze as a 2D array, initialize the solution matrix with zeros, and call the solveMaze function with the starting position (0, 0). If a solution is found, it prints the solution matrix; otherwise, it prints that a solution doesn't exist. Now, let's perform a dry run of the program:

The program starts execution. It calls the solveMaze function with the starting position (0, 0). Inside the solveMaze function: It tries moving right and down from the current cell recursively until the destination (N-1, N-1) is reached or all possible paths are explored. If a solution path is found, it marks the cells in the solution matrix as 1. If the destination is reached, it returns true; otherwise, it backtracks and returns false. Finally, if a solution is found, it prints the solution matrix; otherwise, it prints that a solution doesn't exist.

4 CHAPTER 8

4.1 PRINTING A STRING INPUT AT THE KEYBOARD BACKWARD

```

#include <stdio.h>

// Function to print a string backward recursively
void printStringBackward() {
    char c = getchar();

    // Base case: if newline character is encountered, return
    if (c == '\n') {
        return;
    } else {
        // Recursive case: recursively call printStringBackward and then print the character
        printStringBackward();
        printf("%c", c);
    }
}

```

```

}

int main() {
    printf("Enter a string: ");
    printStringBackward();
    printf("\n");

    return 0;
}

```

EXPLANATION In the `printStringBackward` function, it reads a character from the standard input using `getchar`. The base case checks if the character is a newline character (`'\n'`). If so, it returns, indicating the end of input. Otherwise, it recursively calls itself to read the next character and print it after all characters following it have been printed. In the main function, it prompts the user to enter a string, calls `printStringBackward` function, and then prints a newline character to ensure proper formatting. Now, let's perform a dry run of the program:

The program starts execution. In the main function, it prompts the user to enter a string. Inside the `printStringBackward` function, it reads the first character from the input using `getchar`. It recursively calls itself to read the next character. This process continues until a newline character is encountered. After reaching the base case, it starts printing the characters in reverse order recursively, backtracking from the last character read to the first character read. Finally, in the main function, it prints a newline character to ensure proper formatting.

5 CHAPTER 16

5.1 SELECTION SORT

```

#include <stdio.h>

// Function to swap two elements
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to find the minimum element in the unsorted array
int findMinIndex(int arr[], int start, int end) {
    int minIndex = start;
    for (int i = start + 1; i <= end; i++) {
        if (arr[i] < arr[minIndex]) {

```

```

        minIndex = i;
    }
}
return minIndex;
}

// Function to perform selection sort recursively
void selectionSort(int arr[], int start, int end) {
    if (start >= end) {
        return;
    }
    int minIndex = findMinIndex(arr, start, end);
    swap(&arr[start], &arr[minIndex]);
    selectionSort(arr, start + 1, end);
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    selectionSort(arr, 0, n - 1);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}

```

EXPLANATION The swap function swaps two elements in an array. The findMinIndex function finds the index of the minimum element in the unsorted part of the array from start to end. The selectionSort function performs selection sort recursively. It first finds the minimum element in the unsorted part of the array using findMinIndex, then swaps it with the element at the beginning of the unsorted part, and recursively calls itself with the next index. The printArray

function prints the elements of an array. In the main function, we initialize an array, print the original array, sort it using selection sort, and print the sorted array. Now, let's perform a dry run of the program:

The program starts execution. We initialize an array 64, 25, 12, 22, 11. We print the original array: 64 25 12 22 11. We call the selectionSort function with the array, starting index 0, and ending index 4. Inside the selectionSort function: We find the index of the minimum element in the unsorted part of the array (minIndex = 4 for the first iteration). We swap the minimum element with the element at the beginning of the unsorted part (11 and 64 are swapped). We recursively call selectionSort with the next index (start + 1 = 1) to sort the remaining elements. Inside the recursive call: We find the index of the minimum element in the unsorted part of the array (minIndex = 4 for the second iteration). We swap the minimum element with the element at the beginning of the unsorted part (12 and 25 are swapped). We recursively call selectionSort with the next index (start + 1 = 2) to sort the remaining elements. The process continues until the array is fully sorted. The sorted array is 11, 12, 22, 25, 64.

5.2 QUICK SORT

```
#include <stdio.h>

// Function to swap two elements
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to partition the array
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Choosing the last element as the pivot
    int i = low - 1; // Index of the smaller element

    for (int j = low; j < high; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Function to perform quick sort
void quickSort(int arr[], int low, int high) {
```

```

    if (low < high) {
        // Partition the array and get the pivot index
        int pi = partition(arr, low, high);

        // Recursively sort the subarrays before and after the pivot
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}

```

EXPLANATION The swap function swaps two elements in an array. The partition function chooses the last element as the pivot, rearranges the array such that all elements smaller than the pivot are on the left, and all elements greater than the pivot are on the right. It returns the index of the pivot element after partitioning. The quickSort function sorts the array using quick sort recursively. It first partitions the array using the partition function and then recursively sorts the subarrays before and after the pivot. The printArray function prints the elements of an array. In the main function, we initialize an array, print the original array, sort it using quick sort, and print the sorted array. Now, let's perform a dry run of the program:

The program starts execution. We initialize an array 10, 7, 8, 9, 1, 5. We print the original array: 10 7 8 9 1 5. We call the quickSort function with

the array, starting index 0, and ending index 5. Inside the quickSort function: We partition the array and get the pivot index $pi = 4$ (after partitioning, the array becomes 7, 8, 9, 1, 5, 10). We recursively call quickSort for the subarray before the pivot: quickSort(arr, 0, 3). Inside this recursive call: We partition the subarray and get the pivot index $pi = 2$ (after partitioning, the subarray becomes 7, 8, 1, 9, 5, 10). We recursively call quickSort for the subarray before the pivot: quickSort(arr, 0, 1). Inside this recursive call, since the subarray has only one element, it returns. We recursively call quickSort for the subarray after the pivot: quickSort(arr, 3, 3). Inside this recursive call, since the subarray has only one element, it returns. We recursively call quickSort for the subarray after the pivot: quickSort(arr, 5, 5). Inside this recursive call, since the subarray has only one element, it returns. The sorted array is 1, 5, 7, 8, 9, 10.